

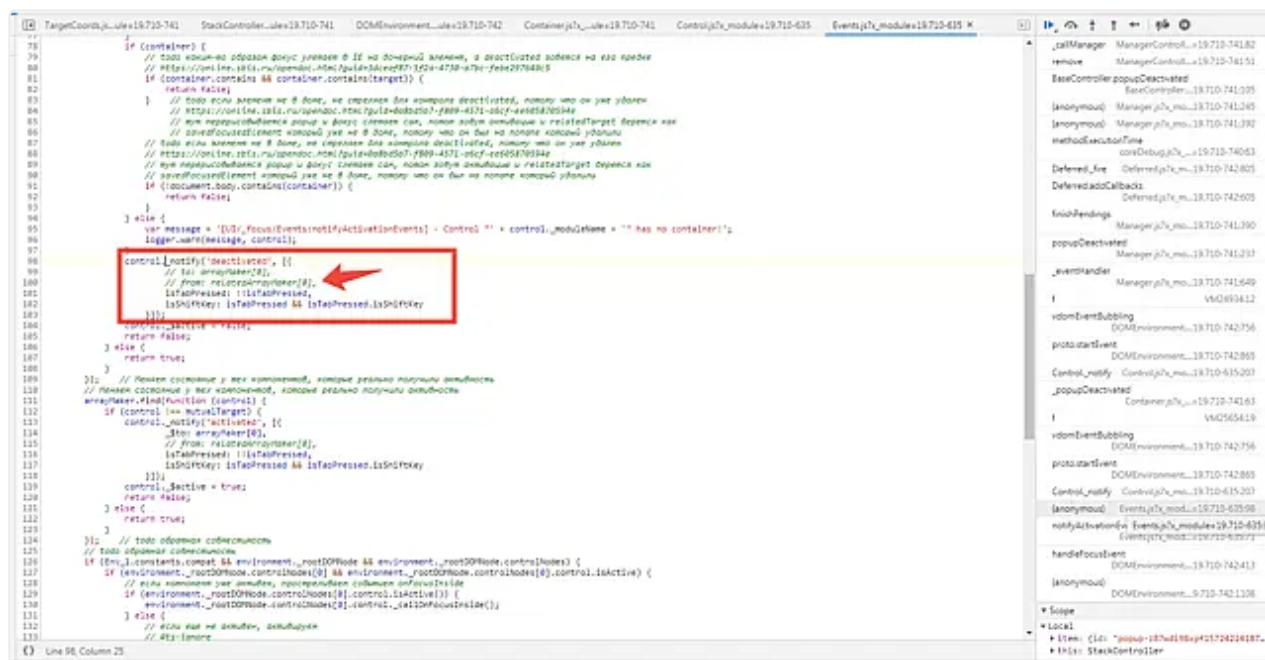
Окно закрывается, не понимаю почему

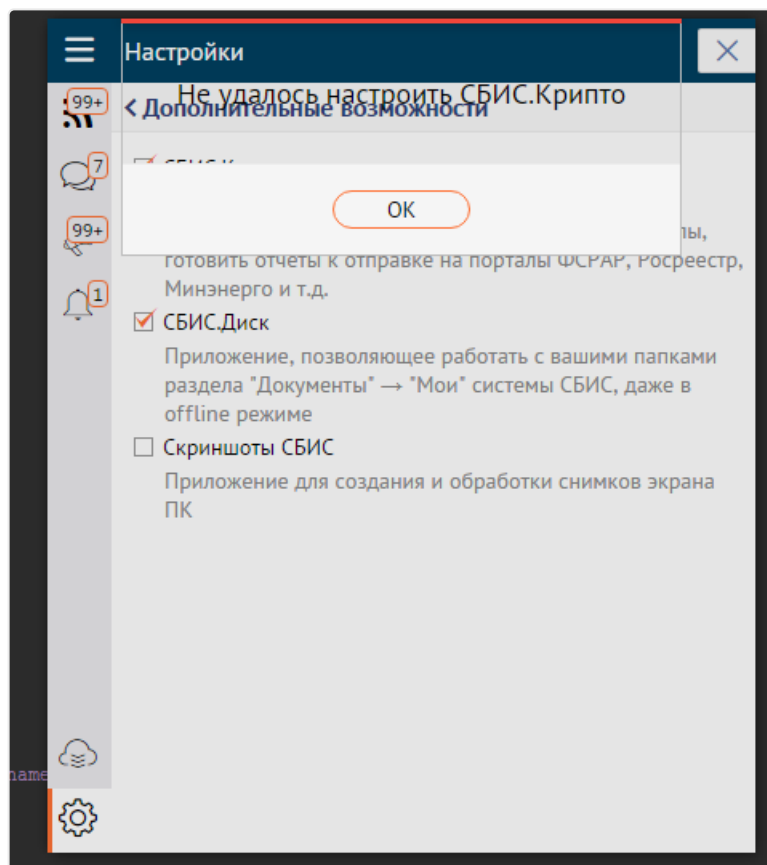
1. Определяем, с каким типом окна работаем, в нашем случае это будет стек. Ставим точку в `elementDestroyed` в файле `"StackController"` и смотрим, что происходит:



1. Сработала деактивация

В случаях когда не все так очевидно, можем посмотреть, куда конкретно ушел фокус. "Events.js" это позволяет сделать.





1. Используется диалоговое окно, значит нам понадобится DialogController. Ставим точку в elementCreated, проверяем конфигурацию:

```

12 width: document.body.clientWidth,
13 height: document.body.clientHeight,
14 scrollTop: 0,
15 };
16
17 // Positioning relative to body
18 item.position = DialogStrategy._getPosition(windowData, sizes, item);
19 _private.fixCompatiblePosition(item);
20
21 fixCompatiblePosition: function(item) {
22 //COMPATIBLE: for
23 if (cfg.popupOptions) {
24 cfg.position.top =
25 cfg.position.left =
26 }
27 }
28 };
29
30 /**
31 * Справка по настройкам
32 * @class Controls/Popup
33 * @control
34 * @private
35 * @category Popup
36 * @extends Controls/Popup
37 */
38 var DialogController = {
39 elementCreated: function(cfg, container) {
40 this.prepareConfig(cfg, container);
41 },
42
43 elementUpdated: function(cfg, container) {
44 /* start: Снимаем установленные значения, влияющие на размер и позиционирование, чтобы получить размеры контента */
45 var width = container.style.width;

```

На первый взгляд все правильно, проваливаемся по F11 и ищем, где заполняется `item.position` (вызывается метод `getPosition` от стратегии).

2. Далее видим, что для правильного позиционирования окна по центру страницы контроллер вычисляет размеры этого окна. Опять же, похоже на правду.

```

15     },
16     },
17     popupDragStart: function(item, container, offset) {
18         if (!item.startPosition) {
19             Object
20                 height: 148
21                 ► margins: {top: 0, left: 0}
22                 width: 366
23             }
24             ► __proto__: Object
25         }
26         offset.x;
27         set.y;
28         //
29         change when you move
30     },
31     },
32     popupDragEnd: function(item, container) {
33         var sizes = this._getPopupSizes(cfg, container); sizes = {width: 366, height: 148, margins: {}}
34         cfg.sizes = sizes;
35         _private.prepareConfig(cfg, sizes);
36     },
37     },
38     },
39     },
40     },
41     },
42     },
43     },
44     },
45     },
46     },
47     },
48     },
49     },
50     },
51     },
52     },
53     },
54     },
55     },
56     },
57     },
58     },
59     },
60     },
61     },
62     },
63     },
64     },
65     },
66     },
67     },
68     },
69     },
70     },
71     },
72     },
73     },
74     },
75     },
76     },
77     },
78     },
79     },
80     },
81     },
82     },
83     },
84     },
85     },
86     },
87     },
88     },
89     },
90     },
91     },
92     },
93     },
94     },
95     },
96     },
97     },
98     },
99     },
100    },

```

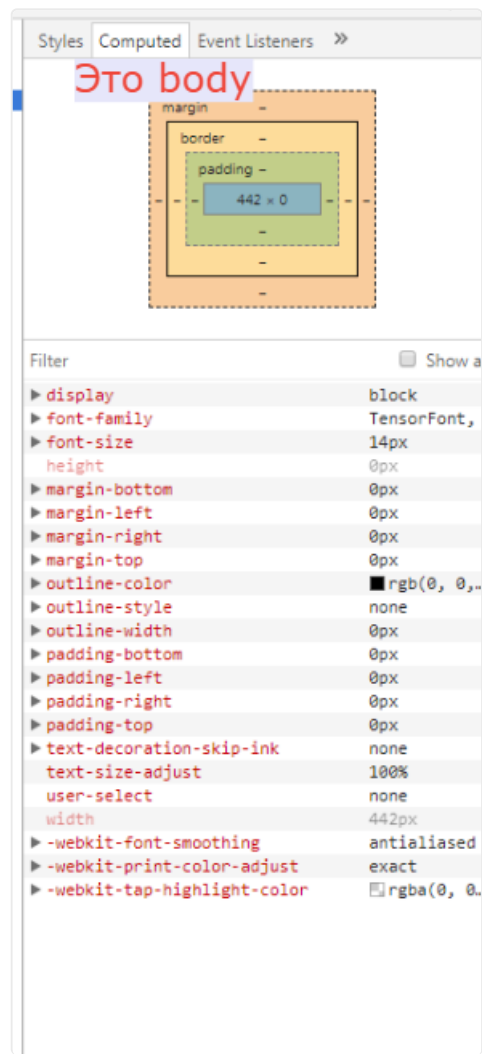
3. Далее мы видим, что в `getPosition` отдаются размеры страницы. И тут уже заметна явная проблема — высота страницы `Orx`.

```

1  (index) (index) require-min.js app-init.js tclosure.js DOMEnvironment.js CssLo
2  define('Controls/Popup/Opener/Dialog/DialogController',
3  [
4      'Controls/Popup/Opener/BaseController',
5      'Controls/Popup/Opener/Dialog/DialogStrategy'
6  ],
7  function(BaseController, DialogStrategy) {
8      var _private = {
9          prepareConfig: function(item, sizes) { item =
10              // After popup will be transferred to the system
11              // we need to return the calculation of the size
12              var windowData = { windowData: {width: 442
13                  width: document.body.clientWidth,
14                  height: document.body.clientHeight,
15                  scrollTop: 0,
16              }
17          };
18          // Positioning relative to body
19          item.position = DialogStrategy._getPosition(windowData, sizes, item); item = {id: "popup-zxuci68rgoq154503016407
20          _private.fixCompatiblePosition(item);
21      },
22      fixCompatiblePosition: function(cfg) {
23          //COMPATIBLE: for old windows user can set the coordinates relative to the body
24          if (cfg.popupOptions.top && cfg.popupOptions.left) {
25              cfg.position.top = cfg.popupOptions.top;
26              cfg.position.left = cfg.popupOptions.left;
27          }
28      }
29  }
30  }

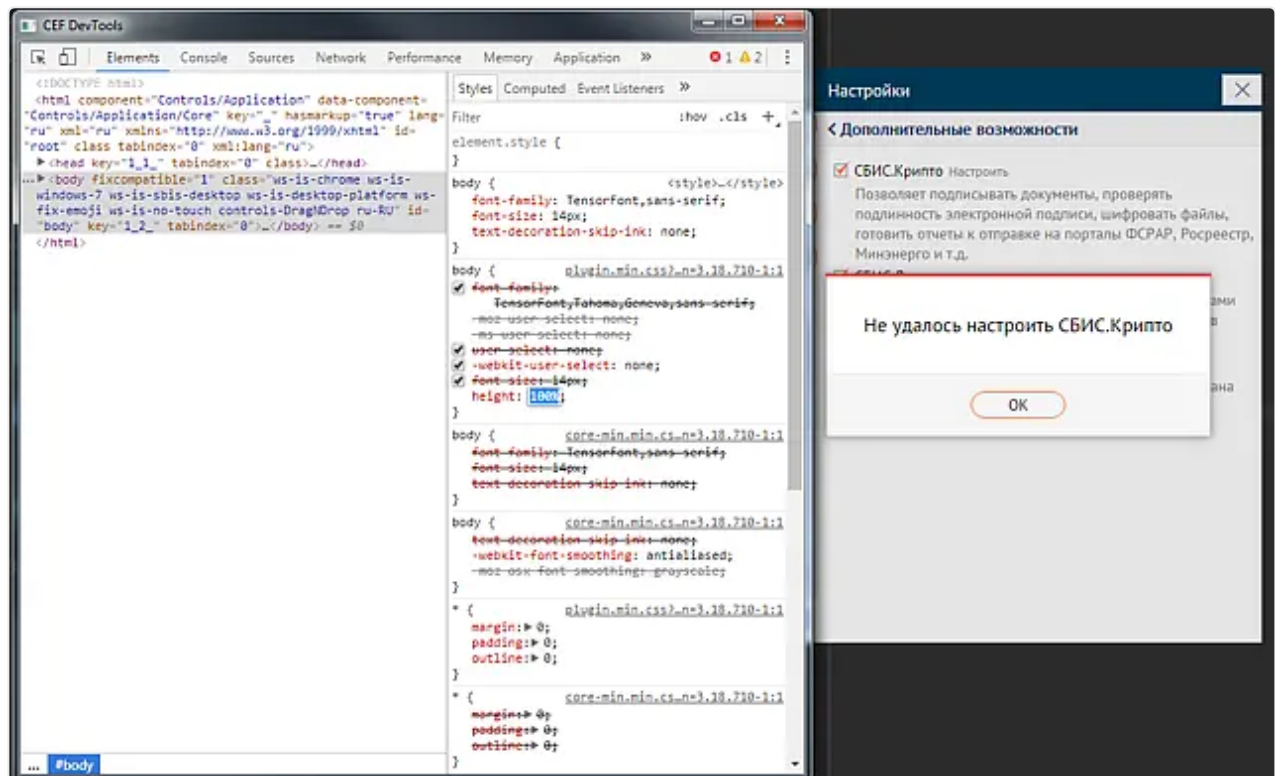
```

Смотрим верстку и видим, что с BODY что-то пошло не так:



Проблема в том, что на BODY не задан размер.

4. Прописываем размер, проверяем:



Ошибка поправлена, все работает.

Отладка в режиме совместимости

Что происходит при открытии панели/окна через action открытия на VDOM странице?

После вызова метода execute нас интересует метод _createComponent в модуле SBIS3.CONTROLS/Action/Mixin/DialogMixin.

```
131     return cMerge(config, meta.componentOptions || {});
132 },
133
134     _createComponent: function(config, meta) {
135         var componentName = this._getComponentName(meta),
136             self = this;
137
138         if (this._isNeedToRedrawDialog()) {
139             this._reloadTemplate(config);
140         } else {
141             this._isExecuting = true;
142             requirejs([componentName], function(Component) { Component = f.constructFn(cfg)
143                 try { true
144                     var deps = []; deps = []
145                     if (!isNewEnvironment()) {
146                         config.mode = meta.mode;
147                         deps = ['Controls/Popup/Opener/BaseOpener', 'Controls/Popup/Compatible/Layer'];
148                         if (meta.mode !== 'dialog' && config.isStack === true) {
149                             deps.push('Controls/Popup/Opener/Stack/StackController');
150                             config._type = 'stack';
151                             config.className = (config.className || '') + ' controls-stack';
152                         } else if (meta.mode !== 'dialog' && config.isStack === false && config.target) {
153                             deps.push('Controls/Popup/Opener/Sticky/StickyController');
154                             config._type = 'sticky';
155                         } else {
156                             deps.push('Controls/Popup/Opener/Dialog/DialogController');
157                             config._type = 'dialog';
158                         }
159                         deps.push(config.template);
160                         requirejs(deps, function(BaseOpener, CompatibleLayer, Strategy, cfgTemplate) {
161                             CompatibleLayer.load().addCallback(function() {
162                                 config._initCompoundArea = function(compoundArea) {
163                                     self._dialog = compoundArea;
164                                 };
165                                 BaseOpener.showDialog(cfgTemplate, config, Strategy);
166                             });
167                         });
168                     } else {
169                         // ...

```

isNewEnvironment() === true говорит нам о том, что мы находимся на VDOM странице и будем открывать новые окна со старыми шаблонами. Далее в строках 148-159 идет определение того, какое именно окно мы будем открывать: Stack, Sticky или Dialog. В этом участке проверяем, что открывается нужный контрол, соответствующий решаемым задачам. Поведение указанных контролов описано в [документации](#).

После загрузки указанных модулей и слоев совместимости вызывается метод showDialog() модуля Controls/Popup/Opener/BaseOpener. Открытие любых окон и панелей, через какой бы action не открывалось окно, проходит через BaseOpener.showDialog. В методе также идет проверка на окружение, и в нашем случае вызывается метод _prepareConfigForOldTemplate, который вставляет слой совместимости между новым попапом и старым шаблоном.

```

152     }
153   });
154   Base.showDialog = function(rootTpl, cfg, controller, popupId, opener) {
155     var def = new Deferred();
156
157     if (Base.isNewEnvironment()) {
158       if (Base.isVDOMTemplate(rootTpl) && !(cfg.templateOptions && cfg.templateOptions._initCompoundArea)) {
159         if (popupId) {
160           popupId = ManagerController.update(popupId, cfg);
161         } else {
162           popupId = ManagerController.show(cfg, controller);
163         }
164         def.callback(popupId);
165       } else {
166         requirejs(['Controls/Popup/Compatible/BaseOpener'], function(CompatibleOpener) { CompatibleOpener = { _prepa
167           CompatibleOpener._prepareConfigForOldTemplate(cfg, rootTpl);
168           if (popupId) {
169             popupId = ManagerController.update(popupId, cfg);
170           } else {
171             popupId = ManagerController.show(cfg, controller);
172           }
173           def.callback(popupId);
174         });
175       }
176     } else {
177       function isFormController() {
178         var proto = rootTpl.prototype && rootTpl.prototype.__proto__;
179         while (proto) {
180           if (proto._moduleName === 'SBIS3.CONTROLS/FormController') {
181             return true;
182           }
183           proto = proto.__proto__;
184         }
185         return false;
186       }
187
188       var deps = ['Controls/Popup/Compatible/BaseOpener'];
189
190       if (isFormController()) {
191         deps.push('SBIS3.CONTROLS/Action/List/OpenEditDialog');
192       } else {
193         // ...

```

Попадая в строку 171 в переменную `cfg` мы имеем конечный конфиг, с которым будет открываться окно. Здесь мы проверяем, что конфигурация соответствует той, что отдавалась в метод открытия; опции диалога соответствуют API новых окон.

The screenshot shows a code editor with a JavaScript file. Line 171 is highlighted, showing the assignment of `popupId` to `ManagerController.show(cfg, controller)`. Below the code editor, the console log displays the `popupOptions` object. The log is filtered to show only the `popupOptions` object. Red annotations highlight specific parts of the log:

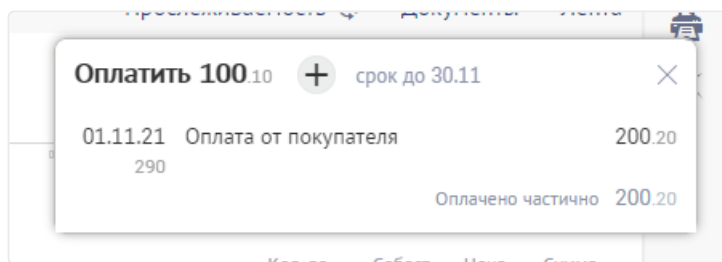
- A red box highlights the `popupOptions` object in the console log, with a red annotation: **Опции слоя совместимости, который выступает в роли старой floatArea для шаблона**.
- Another red box highlights the `templateOptions` property within the `popupOptions` object, with a red annotation: **Опции шаблона**.

Если на этом этапе что-то пошло не так, то с большой вероятностью в конфигурации, переданной при открытии, были допущены ошибки. Перепроверяем ее и, если проблему найти не удалось, то оформляем ошибку на платформу со скриншотами конфигурации контрола, через который открывается окно, и скриншотом конфигурации, которая попадает в метод `show` (см. скриншот выше).

Если в ходе отладки ошибки возникли какие-либо вопросы по слою совместимости, то оформляем их в группе [Wasaby Framework](#) в разделе обсуждения, прикладывая скрины конфигурации контрола, через который открывается окно и конечной конфигурации, которая попадает в метод `show`.

Контент выходит за пределы диалогового окна

Ошибка: Визуально заметно, что уголки внутреннего контента выступают за пределы диалогового окна, тем самым ломая скругления.



Скорее всего, вы задаете цвет фона на одном из дочерних элементов. Если это так, то данный баг вызван нативным поведением браузера (если на родителе заданы скругления, при этом ребенок имеет стиль, определяющий фоновый цвет, уголки выйдут за пределы родительского контейнера).

Для решения проблемы необходимо задать css-класс со свойством **overflow: hidden** на корне своего контента.

